

Análise e Implementação de um Servidor de Streaming de Vídeo via Protocolos RTSP/RTP com Camada de Segurança

Cibele Vale¹, Gabrielle Carvalho¹, Pedro Lucas Santos¹, Samuel Cesar¹, Vitória Matos¹

¹Departamento de Ciências Exatas e da Terra – Universidade do Estado da Bahia (UNEB)
Salvador – BA – Brasil

{cibelevale.estudante, gabriellecarvalhotech, pedrolucas.js18}@gmail.com

{samueljc003, vitorianascimentomatos}@gmail.com

Abstract. *This paper analyzes the development of a real-time video streaming server using RTSP and RTP protocols. The research focuses on encapsulating JPEG video frames into UDP segments, incorporating sequence numbering and timestamps for temporal integrity. Additionally, the project implements a security layer via Fernet encryption and Quality of Service (QoS) through frame rate management. Finally, it provides a technical-commercial outlook on the Brazilian streaming market for 2026.*

Resumo. *Este artigo detalha o desenvolvimento de um servidor de streaming de vídeo em tempo real baseado nos protocolos RTSP e RTP. O foco reside no encapsulamento de quadros de vídeo JPEG em segmentos UDP, incorporando numeração de sequência e marcas de tempo para integridade temporal. O projeto implementa ainda uma camada de segurança via criptografia Fernet e QoS através da gestão de cadência de quadros. Conclui-se com uma análise técnico-comercial do mercado de streaming no Brasil em 2026.*

1. Introdução

O advento da computação e a expansão das redes de banda larga transformaram o streaming de vídeo num dos pilares do tráfego global de dados. No entanto, a transmissão de conteúdo multimídia em tempo real, onde a latência é um fator crítico, impõe desafios técnicos que divergem fundamentalmente do paradigma de transferência de arquivos convencionais. Enquanto o protocolo HTTP, baseado em TCP, prioriza a integridade absoluta dos dados em detrimento do tempo de entrega, aplicações de tempo real exigem uma abordagem que privilegie a fluidez e a sincronização temporal.

Neste contexto, os protocolos RTSP (Real Time Streaming Protocol) e RTP (Real-time Transport Protocol) emergem como padrões fundamentais. O RTSP atua como uma ponte de controle, permitindo a manipulação da sessão (reprodução, pausa e encerramento), enquanto o RTP fornece os mecanismos necessários para o transporte de dados sensíveis ao tempo, como áudio e vídeo, sobre o protocolo UDP.

Este trabalho apresenta a análise e construção de um servidor de streaming que realiza o encapsulamento de quadros de vídeo em pacotes RTP. A investigação não se limita apenas ao transporte bruto; são implementadas camadas de Qualidade de Serviço (QoS) para mitigação de *jitter* e uma camada de segurança baseada em criptografia de ponta-a-ponta. O artigo estrutura-se detalhando a arquitetura de rede, os mecanismos de

encapsulamento, a análise de segurança e, por fim, uma discussão sobre o mercado de streaming brasileiro projetado para o ano de 2026.

2. Arquitetura do Servidor RTSP/RTP

A arquitetura implementada baseia-se em um modelo de separação de canais. O controle da sessão (Play, Pause, Teardown) é realizado via TCP, enquanto a mídia trafega via UDP para priorizar a velocidade.

A utilização de múltiplas threads permitiu a execução paralela das tarefas de sinalização RTSP, transmissão de vídeo e transmissão de áudio, reduzindo bloqueios e aumentando a responsividade da aplicação.

O sistema foi dividido em três componentes principais: cliente, servidor e módulo utilitário. O servidor é responsável por capturar os quadros do vídeo, encapsulá-los em pacotes RTP e transmiti-los ao cliente. O cliente, por sua vez, realiza a recepção dos pacotes, descriptografa os dados e reproduz simultaneamente áudio e vídeo. Já o módulo utilitário concentra parâmetros compartilhados, como portas, chave criptográfica e definição do cabeçalho RTP.

2.1. Máquina de Estados e Sinalização (RTSP)

O servidor gerencia a conexão através da classe `ServerState`, que transita entre os estados `INIT`, `READY` e `PLAYING` conforme as requisições do cliente na porta 8554.

A troca de mensagens RTSP ocorre através de um socket TCP persistente. O cliente envia comandos textuais simples como `SETUP`, `PLAY`, `PAUSE` e `TEARDOWN`, permitindo controlar remotamente a sessão multimídia. Quando o servidor recebe o comando `SETUP`, ele cria as threads responsáveis pelo envio do fluxo de vídeo e áudio.

```
def handle_rtsp(conn, addr):
    print(f"Conexão RTSP de {addr}")
    while True:
        data = conn.recv(1024).decode()
        if not data: break
        request = data.split(' ')[0]

        if request == "SETUP":
            ServerState.state = ServerState.READY
            conn.send(b"RTSP/1.0 200 OK\nSession: 123456")

            arquivo = "hobi67.mp4"

            threading.Thread(target=video_stream_worker, args=(arquivo, addr[0])).start()
            threading.Thread(target=audio_stream_worker, args=(arquivo, addr[0])).start()

        elif request in ["PLAY", "PAUSE", "TEARDOWN"]:
            if request == "PLAY": ServerState.state = ServerState.PLAYING
            elif request == "PAUSE": ServerState.state = ServerState.READY
            elif request == "TEARDOWN": ServerState.state = ServerState.INIT
            conn.send(b"RTSP/1.0 200 OK")
            if request == "TEARDOWN": break
```

Figura 1. Trecho do código responsável pela máquina de estados RTSP do servidor.

A utilização de uma máquina de estados simplifica o gerenciamento da sessão e evita transmissões indevidas. Por exemplo, quando o estado está definido como `READY`, as threads permanecem ativas, porém sem transmitir mídia, aguardando um comando `PLAY`. Já no estado `PLAYING`, os fluxos RTP passam a ser enviados continuamente.

2.2. Encapsulamento de Dados (RTP)

Cada quadro de vídeo capturado via OpenCV é convertido para o formato JPEG para reduzir a carga de rede. O payload resultante é encapsulado na classe `RTPPacket`, que adiciona um cabeçalho de 12 bytes contendo:

- **Número de Sequência:** Para detecção de pacotes perdidos ou fora de ordem.
- **Timestamp:** Baseado em uma frequência de 90kHz para sincronização temporal da reprodução.
- **Payload Type:** Identificado como JPEG.

O encapsulamento manual permitiu compreender de forma prática o funcionamento interno do protocolo RTP. O cabeçalho foi montado utilizando a biblioteca `struct`, responsável por converter os dados para o formato binário adequado antes da transmissão.

O vídeo é transmitido utilizando sockets UDP independentes do áudio, reduzindo atrasos e permitindo uma reprodução mais fluida. Embora o UDP não ofereça garantia de entrega, ele é amplamente utilizado em aplicações multimídia por apresentar baixa latência.

Além do fluxo de vídeo, o sistema também implementa transmissão de áudio. A trilha sonora é extraída do arquivo MP4 através da biblioteca `moviepy`, convertida para PCM 16 bits e enviada em pequenos blocos para reprodução contínua no cliente utilizando `PyAudio`.

```
class RTPPacket:
    def __init__(self, payload_type, seq_num, timestamp, payload):
        self.version = 2
        self.payload_type = payload_type
        self.seq_num = seq_num
        self.timestamp = timestamp
        self.payload = payload

    def get_packet(self):
        # Cabeçalho RTP simplificado
        header = struct.pack('!BBHII',
                             (self.version << 6),
                             self.payload_type,
                             self.seq_num,
                             self.timestamp,
                             0) # SSRC ou Identificador
        return header + self.payload
```

Figura 2. Trecho do código mostrando a classe `RTPPacket`

Como ilustrado na Figura 2, a classe `RTPPacket` foi responsável pela criação manual dos pacotes RTP utilizados na transmissão do fluxo multimídia. O cabeçalho foi estruturado contendo informações essenciais para sincronização e controle da reprodução, como número de sequência, timestamp e tipo do payload.

A implementação manual desse mecanismo permitiu compreender de forma prática o funcionamento interno do protocolo RTP e sua importância em aplicações de streaming em tempo real. Além disso, o encapsulamento dos dados tornou possível separar adequadamente o cabeçalho das informações multimídia transmitidas via UDP.

3. Segurança e QoS (Qualidade de Serviço)

3.1. Criptografia de Fluxo

Para garantir a privacidade da transmissão, o sistema utiliza a biblioteca `cryptography` com o esquema Fernet. Todo pacote RTP tem seu conteúdo cifrado antes de ser enviado

ao socket UDP, sendo decifrado apenas no cliente que possui a chave compartilhada KEY.

A criptografia é aplicada diretamente sobre o payload do pacote RTP, impedindo que terceiros interpretem o conteúdo transmitido mesmo que consigam interceptar os datagramas UDP na rede. Esse mecanismo adiciona uma camada importante de confidencialidade ao sistema.

No cliente, a descriptografia ocorre imediatamente após o recebimento do pacote. Somente após a validação e decodificação dos dados o quadro é reconstruído via OpenCV ou reproduzido no dispositivo de áudio.

```
while cap.isOpened():
    if ServerState.state == ServerState.PLAYING:
        start_time = time.time()

        ret, frame = cap.read()
        if not ret: break

        frame = cv2.resize(frame, (640, 480))
        _, buffer = cv2.imencode('.jpg', frame, [cv2.IMWRITE_JPEG_QUALITY, 50])

        payload = fernet.encrypt(buffer.tobytes())
        timestamp = int(time.time() * 90000) & 0xFFFFFFFF
        pacote = RTPPacket(26, seq_num, timestamp, payload)
        video_socket.sendto(pacote.get_packet(), (ip_destino, PORTA_RTP_VIDEO))
```

Figura 3. Processo de criptografia do payload RTP utilizando Fernet no servidor.

A Figura 3 apresenta o processo de criptografia aplicado ao payload antes do envio pela rede. O servidor utiliza o método `fernet.encrypt()` para cifrar os dados do quadro de vídeo ou do bloco de áudio, garantindo confidencialidade durante a transmissão via UDP.

Esse mecanismo impede que terceiros interpretem o conteúdo transmitido caso os pacotes sejam interceptados na rede. A utilização do esquema Fernet também oferece integridade dos dados, assegurando que o conteúdo não seja alterado durante o tráfego.

```
def receive_video(self):
    video_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    video_socket.bind(('0.0.0.0', PORTA_RTP_VIDEO))
    while not self.stop_rtp:
        try:
            data, _ = video_socket.recvfrom(65535)
            payload = fernet.decrypt(data[12:])
            frame = cv2.imdecode(np.frombuffer(payload, np.uint8), cv2.IMREAD_COLOR)
            if frame is not None:
                cv2.imshow('Video', frame)
                cv2.waitKey(1)
        except: continue

def receive_audio(self):
    audio_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    audio_socket.bind(('0.0.0.0', PORTA_RTP_AUDIO))
    while not self.stop_rtp:
        try:
            data, _ = audio_socket.recvfrom(8192)
            self.audio_stream.write(fernet.decrypt(data))
        except: continue
```

Figura 4. Processo de descriptografia realizado pelo cliente após o recebimento dos pacotes RTP.

Conforme ilustrado na Figura 4, o cliente realiza a descriptografia utilizando o método `fernet.decrypt()` imediatamente após o recebimento do pacote UDP. Após esse processo, os dados originais são reconstruídos e encaminhados para decodificação via OpenCV ou reprodução através do PyAudio.

A implementação da descryptografia no lado do cliente garantiu que apenas dispositivos autorizados, possuindo a chave compartilhada `KEY`, fossem capazes de acessar corretamente o conteúdo multimídia transmitido.

3.2. Mecanismos de QoS

O servidor implementa QoS através do controle de fluxo baseado no FPS (*Frames Per Second*) nativo do vídeo. O cálculo do `sleep_time` garante que o servidor não sobrecarregue a banda do cliente e mantenha a cadência correta da mídia.

O controle temporal do streaming é realizado através do cálculo dinâmico do tempo de envio de cada quadro do vídeo. O servidor mede o tempo necessário para capturar, processar, encapsular e transmitir cada frame, utilizando esse valor para determinar o intervalo ideal antes da transmissão seguinte.

Com base no FPS original do vídeo, o sistema calcula automaticamente um tempo de espera (`sleep_time`) entre os envios dos pacotes RTP. Esse mecanismo permite manter a cadência correta da reprodução, evitando sobrecarga da rede e reduzindo problemas como atrasos, jitter e travamentos perceptíveis durante a execução do fluxo multimídia.

Esse mecanismo reduz problemas de sincronização, diminui o consumo excessivo de rede e evita que o cliente receba quadros mais rapidamente do que consegue processar. Na prática, isso contribui para reduzir fenômenos como jitter e travamentos perceptíveis durante a reprodução.

Além disso, a separação entre threads de áudio e vídeo permite maior paralelismo e melhora a responsividade do sistema durante comandos RTSP, como pausa e encerramento da sessão.

4. Resultados Obtidos

Durante os testes realizados em ambiente local, o sistema conseguiu transmitir vídeo e áudio simultaneamente com baixa latência e estabilidade satisfatória. O cliente foi capaz de reproduzir os quadros recebidos em tempo real utilizando OpenCV, enquanto o áudio foi reproduzido continuamente através do PyAudio.

A utilização do UDP demonstrou eficiência na redução de atrasos de transmissão, embora ocasionalmente possam ocorrer perdas de pacotes devido à ausência de mecanismos de retransmissão. Mesmo assim, a reprodução manteve-se funcional e fluida na maior parte do tempo.

Os testes também demonstraram o funcionamento correto da máquina de estados RTSP. Os comandos de pausa, reprodução e encerramento foram interpretados adequadamente pelo servidor, alterando dinamicamente o comportamento das threads de streaming.

A camada de criptografia Fernet mostrou-se eficiente para proteger os dados trafegados, garantindo confidencialidade sem comprometer significativamente o desempenho da aplicação.

Durante os testes de transmissão, foi observado que a resolução do vídeo impactava diretamente no desempenho da aplicação e no tamanho dos pacotes enviados pela rede. Quanto maior a resolução utilizada, maior era o volume de dados encapsulados nos pacotes RTP, o que aumentava significativamente a carga de processamento e dificultava a

transmissão contínua para o cliente. Em alguns cenários, vídeos em resoluções mais altas não puderam ser transmitidos corretamente devido ao tamanho excessivo dos pacotes e à limitação de processamento em tempo real.

Além disso, foi realizado um monitoramento do consumo de CPU durante a execução do servidor. Com apenas um cliente conectado reproduzindo vídeo, o processamento permaneceu em aproximadamente 30% da CPU. Entretanto, ao adicionar um segundo cliente simultaneamente, o consumo subiu para quase 60%, evidenciando o impacto do paralelismo, da criptografia e do processamento multimídia sobre os recursos computacionais da máquina. Esses resultados demonstram como aplicações de streaming em tempo real exigem otimizações de compressão, gerenciamento de threads e controle de qualidade de serviço para manter estabilidade sob múltiplas conexões.

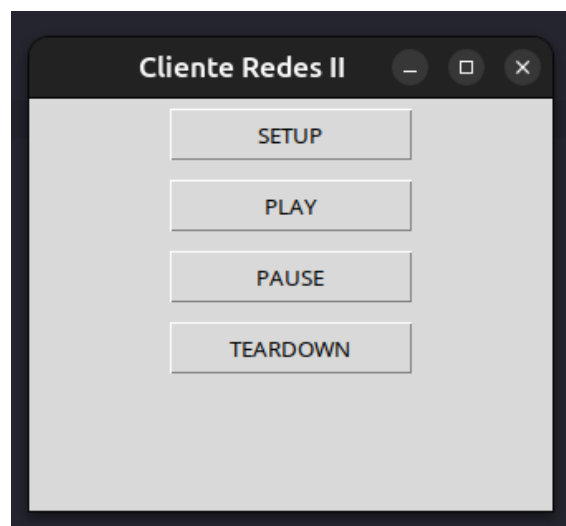


Figura 5. Interface gráfica do cliente RTSP desenvolvida com Tkinter.

A Figura 5 apresenta a interface gráfica desenvolvida para o cliente RTSP utilizando a biblioteca Tkinter. O menu foi projetado de forma simples e objetiva, permitindo controlar remotamente a sessão de streaming através dos comandos RTSP fundamentais.

O botão `SETUP` é responsável por iniciar a configuração da sessão entre cliente e servidor. Ao ser acionado, o cliente estabelece a conexão RTSP via TCP e inicializa as threads responsáveis pela recepção dos fluxos de áudio e vídeo.

O comando `PLAY` altera o estado do servidor para `PLAYING`, iniciando efetivamente a transmissão dos pacotes RTP contendo os dados multimídia. Durante esse estado, os fluxos de vídeo e áudio passam a ser enviados continuamente ao cliente.

O botão `PAUSE` interrompe temporariamente a transmissão sem encerrar a sessão RTSP. Nesse caso, o servidor retorna ao estado `READY`, mantendo as threads ativas, porém suspendendo o envio dos pacotes RTP até que um novo comando `PLAY` seja recebido.

Por fim, o comando `TEARDOWN` encerra completamente a sessão de streaming, finalizando as transmissões RTP, interrompendo as threads de recepção e encerrando a interface gráfica do cliente.

5. Análise do Mercado de Streaming

Em 2026, o mercado brasileiro de streaming atingiu um estágio de alta maturidade técnica e consolidação comercial. Impulsionado pela expansão maciça das redes de fibra óptica e pela cobertura quase universal do 5G nos grandes centros urbanos, o consumo de vídeo sob demanda (VOD) e transmissões ao vivo tornou-se o principal responsável pelo tráfego de dados no país. Nesse cenário, serviços como Netflix, Prime Video, Disney+ e Globo-play não apenas dominam a rede nacional, mas também ditam os padrões tecnológicos e de consumo da indústria.

5.1. Arquitetura Técnica: Padrões Comerciais vs. Escopo do Projeto

Para suportar milhões de usuários simultâneos sem gargalos, as gigantes do streaming utilizam arquiteturas altamente descentralizadas baseadas em Redes de Fornecimento de Conteúdo (CDNs).

- **Protocolos Adaptativos (DASH e HLS):** Diferente do RTSP (Real-Time Streaming Protocol) abordado neste projeto, que estabelece uma sessão contínua e stateful entre cliente e servidor, as plataformas comerciais operam de forma stateless utilizando HTTP. Protocolos como MPEG-DASH e Apple HLS fragmentam o vídeo em pequenos segmentos (chunks) de poucos segundos. Isso permite a Adaptive Bitrate Streaming (ABR), onde o reprodutor do cliente avalia a largura de banda disponível e solicita, dinamicamente, segmentos em maior ou menor resolução, evitando o buffering.
- **Adoção do HTTP/3 (QUIC):** Em 2026, o transporte subjacente consolidou-se em torno do HTTP/3, baseado no protocolo QUIC (desenvolvido sobre UDP). Essa mudança reduziu drasticamente a latência na conexão inicial e eliminou o problema de Head-of-Line Blocking presente no TCP (usado no HTTP/2), garantindo transmissões mais fluidas mesmo em redes móveis instáveis.
- **Contraste com o RTSP:** Embora o mercado comercial tenha migrado para soluções baseadas em HTTP para escalar via CDNs tradicionais, o estudo e a implementação do RTSP mantêm-se altamente relevantes. O RTSP oferece uma latência ponta a ponta significativamente menor, sendo o padrão ouro para cenários que exigem tempo real rigoroso, como monitoramento de segurança (câmeras IP), robótica remota e transmissões em redes locais corporativas.

5.2. Dinâmicas Comerciais e Modelos de Negócio

Com o esgotamento do crescimento orgânico de novos assinantes no Brasil, a concorrência em 2026 não é mais apenas pela aquisição, mas pela retenção de usuários. Para sustentar a lucratividade, as estratégias comerciais sofreram mutações significativas:

- A fragmentação excessiva de plataformas gerou a fadiga das assinaturas no consumidor. Como resposta, o mercado consolidou o modelo de Bundles. Operadoras de telecomunicações, bancos e marketplaces passaram a atuar como hubs agregadores, oferecendo pacotes que combinam múltiplas plataformas por um valor único e com desconto agressivo, garantindo contratos de maior fidelidade.

- O modelo puro de assinatura premium cedeu espaço a planos mais acessíveis subsidiados por publicidade. A adoção de anúncios tornou-se um padrão vital de receita. Tecnologias de Server-Side Ad Insertion (SSAI) são utilizadas para costurar o anúncio diretamente no fluxo de vídeo, burlando bloqueadores de anúncios e oferecendo publicidade hiper-direcionada com base no perfil de consumo do usuário.
- No cenário brasileiro, a Globoplay se destaca por aliar a tecnologia de streaming a um sistema massivo de conteúdo, dominando transmissões ao vivo de esportes e o filão de novelas, o que força as concorrentes estrangeiras a investirem pesadamente em produções nacionais para se manterem competitivas.

5.3. Comparação entre o FTP e os serviços de Streaming da Netflix

O sistema FTP desenvolvido neste trabalho possui semelhanças importantes com os serviços de streaming utilizados pela Netflix, pois ambos dependem da comunicação entre cliente e servidor para realizar a transferência de dados através da rede. No caso do FTP, os arquivos são enviados ou recebidos de forma completa, garantindo que todo o conteúdo seja transferido corretamente antes do uso final.

Já a Netflix utiliza técnicas de streaming, nas quais os dados do vídeo são divididos em pequenos segmentos transmitidos continuamente ao usuário, permitindo reprodução em tempo real sem a necessidade de baixar todo o arquivo previamente. Além disso, ambos os sistemas utilizam protocolos de transporte confiáveis para assegurar estabilidade, integridade dos dados e controle da comunicação durante a transmissão. Dessa forma, mesmo com objetivos diferentes, tanto o FTP quanto as plataformas de streaming demonstram a importância das arquiteturas cliente-servidor e dos protocolos de rede para o funcionamento eficiente de aplicações modernas na internet.

6. Conclusão

A implementação demonstrou a viabilidade do uso de sockets UDP para streaming de baixa latência. A adição de cabeçalhos RTP manuais e camadas de criptografia por software provou ser um exercício fundamental para entender as camadas mais baixas da pilha TCP/IP e os desafios de entrega de dados sensíveis ao tempo.

O desenvolvimento do projeto também proporcionou um aprofundamento prático no estudo de bibliotecas amplamente utilizadas em aplicações multimídia e redes de computadores. A utilização do OpenCV permitiu compreender técnicas de captura, compressão e reconstrução de quadros de vídeo em tempo real, enquanto a biblioteca PyAudio possibilitou explorar mecanismos de reprodução contínua de áudio através de buffers PCM.

Além disso, o uso da biblioteca `cryptography` com o esquema Fernet contribuiu para o entendimento de conceitos relacionados à segurança da informação, confidencialidade de dados e criptografia simétrica aplicada em transmissões multimídia. O projeto também exigiu estudo sobre gerenciamento de threads em Python, sincronização entre processos concorrentes e comunicação via sockets TCP e UDP.

A integração entre protocolos RTSP/RTP, processamento multimídia, criptografia e controle temporal permitiu consolidar conhecimentos importantes sobre sistemas

distribuídos e aplicações de tempo real. Dessa forma, o trabalho não apenas demonstrou conceitos teóricos de redes de computadores, mas também proporcionou experiência prática no desenvolvimento de aplicações multimídia completas.

Referências

- [1] Kurose, J. F., & Ross, K. W. (2021). *Redes de Computadores e a Internet*.
- [2] Código Fonte do Projeto: `server.py`, `utils.py` e `client.py`.
- [3] RFC 3550: *RTP: A Transport Protocol for Real-Time Applications*.